API Platform

Table des matières

1. Introduction 2
2. Installation 2
3. Paramétrage 2
3.1 Exposer une entité 3
3.2 Comprendre ce que l'on obtient 3
3.3 Choisir ce que l'on expose 4
3.4 Un petit mot sur le format 4
4. Serialization 5
4.1 Gérer les relations 7
5. Les Filtres 7
6. La Pagination 8

1. Introduction

Maintenant qu'on a bien compris les concepts de base de Symfony lorsqu'il s'agit de créer une WebApp, on va pouvoir sortir un peu de ce cadre.
Rappelez-vous : lorsque vous initialisez un projet Symfony, vous lui précisez le flag **--webapp** dans la commande **symfony new**.

Aujourd'hui nous allons voir quelles différences existent entre l'architecture **webapp** que l'on connaît, et l'architecture **classique** de Symfony.

Voici un tableau récapitulant ce qui est intégré par défaut selon le type de projet que l'on crée :

Composant	Webapp	Classique
Controllers	✅	✅
Entity	✅	✅
Repository	✅	✅
Doctrine	✅	❌ (à installer avec composer)
MakerBundle	✅	❌
TWIG	✅	❌
Fixtures	❌ (à installer avec composer)	❌

On peut voir ici que le flag --**webapp** que l'on utilise dans la commande nous sert en réalité simplement à simplement à intégrer des éléments de Symfony par défaut dans le projet, il nous épargne donc l'installation de Twig, de Doctrine, et du MakerBundle

Mais le cœur de Symfony ce n'est pas ça, c'est bien le trio Entity / Repository / Controller.

Ce trio contient toute la **logique métier** de l'application, et aujourd'hui, nous allons voir comment ce trio nous permet de générer très rapidement une **API complète, documentée et fonctionnelle** en se basant uniquement sur cette structure réduite, et ce grâce à **ApiPlatform**.

Cette API pourra ensuite être consommé par un autre projet de votre choix, disons par exemple, un projet Angular ☺

2. Installation

```
composer require api
```

3. Paramétrage

L'installation que nous avons faite en étape 2 va créer un fichier crucial :

/config/packages/api_platform.yaml

Ce fichier va nous servir à configurer les comportements voulus de l'API, allant du format de retour des données jusqu'à la pagination en passant par le numéro de version… Pour le moment on va le

laisser tel quel, le format de retour de donnée en JSON nous va très bien, c'est le format attendu d'une API REST.

3.1 Exposer une entité

Attention ça va aller très très vite : Nous allons prendre une entité quelconque, par exemple l'entité Brand, et nous allons nous rendre dans le fichier correspondant.

Nous allons rajouter l'annotation suivante sur sa classe :

N'oubliez pas le USE juste au dessus et attention Resource s'écrit à l'anglaise avec un seul S !

```
10  use ApiPlatform\Metadata\ApiResource;
11
12  #[ApiResource]
13  #[ORM\Entity(repositoryClass: BrandRepository::class)]
14  #[ORM\Table(name: 'brand')]
15  class Brand
16  {
17      #[ORM\Id]
```

Maintenant vous allez démarrer votre serveur si ce n'est pas déjà fait, et vous allez aller sur l'url :

https://localhost:8000/api

Oui oui, c'est bien une doc Swagger fonctionnelle générée toute seule avec une annotation. Et attention on n'a pas que la documentation, les endpoints aussi sont fonctionnels ! Enfin presque tous, on a encore un peu de paramétrage à faire pour éviter des boucles infinies du genre Brands qui appellent toutes les Parts, qui eux-même ont une Brand etc… Regardons ça en détail justement :

3.2 Comprendre ce que l'on obtient

Par défaut, API Platform génère 6 **opérations** pour chaque ressource :

Opération	Méthode HTTP	URL	Fonction
GetCollection	GET	/api/brands	Récupérer plusieurs enregistrements
Post	POST	/api/brands	Créer un enregistrement
Get	GET	/api/brands/{id}	Récupérer un seul enregistrement
Patch	PATCH	/api/brands/{id}	Modifier un enregistrement
Delete	DELETE	/api/brands/{id}	Supprimer un enregistrement

Bon à savoir : il existe également une sixième méthode **PUT** qu'on peut lui dire de créer, elle vise à modifier un enregistrement mais en le remplaçant **entièrement**, pas juste un champ, elle n'est donc pas générée par défaut.

3.3 Choisir ce que l'on expose

ApiPlatform génère tous nos endpoints, mais a-t-on besoin de tout ? Je peux vouloir exposer la liste des Brands par exemple ne pas vouloir laisser la possibilité de créer des nouvelles Brands dans ma base. Dans ce cas je n'aurais pas exemple pas besoin de la méthode POST sur l'endpoint /brands.

C'est là que le tableau du point 3.2 va nous être utile

On va s'en servir pour enrichir notre **annotation** du point 3.1 en ne mettant que les **opérations** que vous voulez. Comme ceci :

Encore une fois n'oulbliez pas les **use** pour les nouvelles classes que vous importez

```
use ApiPlatform\Metadata\ApiResource;
use ApiPlatform\Metadata\Get;
use ApiPlatform\Metadata\GetCollection;

#[ApiResource(
    operations: [
        new GetCollection(),
        new Get(),
    ]
)]
#[ORM\Entity(repositoryClass: BrandRepository::class)]
#[ORM\Table(name: 'brand')]
class Brand
{
```

Dans cette exemple, je viens par exemple de passer ma classe **Brand** en **lecture seule**, je n'aurais plus d'endpoint PATCH, DELETE, POST.

Rechargez votre url https://localhost:8000/api et vous verrez le résultat.

3.4 Un petit mot sur le format

Le format par défaut est un format que l'on appelle JSON+LD. C'est du JSON enrichi par des données supplémentaire comme @context etc. C'est un standard d'API rest mais on peut se retrouver embêté si l'on veut faire fonctionner notre projet qui va consommer l'API avec du JSON classique. Pas d'inquiétude, on a deux options pour le faire :

- **OPTION A**, de loin la plus simple : On dit à l'API de renvoyer du format JSON classique par défaut au lieu du JSON+LD. Pour cela on se rend dans /config/packages/api_platform.yaml et on renseigne ceci :

```yaml
config > packages > ! api_platform.yaml
api_platform:
    title: Hello API Platform
    version: 1.0.0
    defaults:
        stateless: true
        cache_headers:
            vary: ['Content-Type', 'Authorization', 'Origin']
    formats:
        json:
            mime_types: ['application/json']
```

- **OPTION B**, on laisse le choix au développeur : On va prévoir plusieurs formats de retour possibles, et dans les headers de votre application qui va émettre la requête en direction de l'API vous préciserez le format souhaité. Pour faire ça on se rend dans le même fichier /config/packages/api_platform.yaml et on renseigne la clé « formats » comme ceci :

```yaml
config > packages > ! api_platform.yaml
api_platform:
    title: Hello API Platform
    version: 1.0.0
    defaults:
        stateless: true
        cache_headers:
            vary: ['Content-Type', 'Authorization', 'Origin']
    formats:
        jsonld:
            mime_types: ['application/ld+json']
        json:
            mime_types: ['application/json']
```

Et dans ce cas, vous devrez préciser le header **Accept: application/json** dans votre requête

4. Serialization

C'est un point crucial. Par défaut, API Platform essaie de sérialiser **toutes les propriétés** de l'entité, y compris les relations — ce qui peut provoquer des boucles infinies (Part → Brand → parts → Part → ...). On va donc utiliser des annotations pour détailler ce qui doit apparaître ou pas selon le scénario.

La solution propre est d'utiliser les **groupes de sérialisation**. On les déclare sur les propriétés avec l'attribut #[Groups] du Serializer Symfony. Prenons un exemple en restant sur l'entité Brand

N’oubliez pas le **use** comme d’habitude

```
#[ApiResource(
    operations: [
        new GetCollection(),
        new Get(),
        new Post()
    ],
    normalizationContext: ['groups' => ['brand:read']],
    denormalizationContext: ['groups' => ['brand:write']]
)]
#[ORM\Entity(repositoryClass: BrandRepository::class)]
#[ORM\Table(name: 'brand')]
class Brand
{
    #[ORM\Id]
```

- **normalizationContext :** décrit ce qui est exposé en **lecture** c’est ce groupe là qui sera géré lorsque l’on sera en **GET**
- **denormalizationContext** : décrit ce qui est accepté en **écriture** (POST, PUT, PATCH)

Pour le moment ça nous avance pas beaucoup d’avoir rajouté ceci mais on n’a pas fini : il va maintenant falloir déclarer quelles sont les propriétés de la classe Brand qui appratiennent à tel ou tel Groupe.

Prenons un exemple : l’ID de ma table Brand n’a aucune vocation à être écrit. Il va être généré par ma base de donnée et ne changera jamais, alors je vais simplement le mettre dans le groupe de lecture mais pas dans celui d’écrire, grâce à cette annotation :

```
class Brand
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    #[Groups(['brand:read'])]
    private ?int $id = null;
```

Le nom de ma Brand en revanche, lui, peut être mis en écriture (à condition que j’accepte les méthode POST ou PATCH), si je veux laisser la main aux utilisateurs pour créer des Brands. Alors j’annote ceci au dessus de la propriété $name :

```
#[Groups(['brand:read', 'brand:write'])]
private ?string $name = null;
```

Maintenant lorsque j'appelle mon endpoint /brands en GET, il ne ressort dans le JSON que les propriétés que j'ai annotées avec le groupe **brand:read**

4.1 Gérer les relations

La serialization est précisément ce qui va nous permettre de gérer les boucles infinies mentionnées plus haut : on va déclarer certains champs d'une entité comme appartenant à un groupe bien précis pour que lorsque l'entité en question est appelée, alors seuls les champs du groupe de cette entité seront appelés. Cas pratique :

- Lorsque qu'on appelle une Brand, elle est renvoyée par l'API avec ses Parts correspondantes
- Sauf que les Parts on également une Brand, il y a donc un risque de boucle infinie. Et même dans l'autre sens d'ailleurs.

Mettons en place ce cas de figure :

Je veux récupérer toutes les pièces auto ainsi que leur marque.

1. Je vais annoter ma classe **Part** pour qu'elle autorise les opération GetCollection() et Get() seulement
2. Je complète mon annotation en rajoutant un groupe pour le **normalizationContext** : ça sera le groupe **part:read**
3. Je rajoute l'annotation **#[Groups(['part:read'])]** sur toutes les propriétés de l'entité **Part** que je vais vouloir récupérer à l'affichage. **Y compris la propriété $brand**
4. Je me rend dans l'entité Brand et je rajoute le groupe **part:read** dans la liste des groupes dont fait partie la propriété **$name**

Et voilà, allez tester sur l'URL de l'API : lorsque vous appelez une **Part**, vous obtenez bien le nom de sa marque, seulement son nom, mais pas le reste, donc pas de boucle infinie.

5. Les Filtres

API Platform propose des filtres prêts à l'emploi. Pour permettre de filtrer les pièces par marque ou par catégorie :

N'oubliez pas le USE pour la classe ApiFilter et SearchFilter

```
#[ApiFilter(SearchFilter::class, properties: [
    'brand' => 'exact',
    'category' => 'exact',
    'name' => 'partial',
])]
class Part
{
    #[ORM\Id]
```

Ça donne automatiquement :

- /api/parts?brand=/api/brands/1 par exemple pour la marque
- /api/parts?category=/api/categories/1 pour rechercher sur la catégorie
- /api/parts?name=filtre pour rechercher sur le **name**

Petite note sur cette url : /api/parts?brand=/api/brands/1

Ici la référence de la Brand N°1 n'est pas N°1, mais bien l'endpoint complet qui permet de l'obtenir. C'est une norme de JSON+LD qui lui permet de fonctionner plus rapidement, mais si on passe l'API en JSON pur alors on pourra demander l'URL /api/parts?brand=1

D'autres filtres utiles : **OrderFilter** pour le tri, **RangeFilter** pour filtrer par plage de prix, **BooleanFilter** pour isAvailable. Je vous laisse chercher ce dont vous aurez besoin ☺

6. La Pagination

Finissons le cœur léger avec un truc très simple pour répondre à une question importante : Combien de résultats d'un coup souhaite-t-on récupérer ? Dans le projet Autopote vous en avez 500, on va peut-être pas tous les afficher d'un coup…

Solution simple :

/config/packages/api_platform.yaml

```
config > packages > ! api_platform.yaml
1   api_platform:
2       title: Hello API Platform
3       version: 1.0.0
4       defaults:
5           pagination_items_per_page: 20
6           stateless: true
7           cache_headers:
8               vary: ['Content-Type', 'Authorization', 'Origin']
9       formats:
10          jsonld:
11              mime_types: ['application/ld+json']
12          json:
13              mime_types: ['application/json']
```

Solution personnalisée si on veut changer le nombre de résultat par page selon les entités :

```
#[ApiResource(
    paginationItemsPerPage: 10,
    operations: [
        new GetCollection(),
        new Get()
    ],
    normalizationContext: ['groups' => ['part:read']],
)]
```

Les deux solutions nous donne accès à la même fonctionnalité : la possibilité de passer un paramètre **page** dans tous nos endpoints, et il s'en débrouille. Exemple : /api/parts?page=2 pour afficher les résultats de 11 à 20.

EXERCICE

- Appliquez ces annotations sur toutes nos entités : Pour le moment, ne mettez les entités qu'en Lecture ! Vous aurez besoin de déclarer les opération Get() et GetCollection(), c'est tout

- Annotez les propriétés de vos classes pour que tous les endpoints fonctionnent sans créer de boucles infinies : Utilisez les Groupes pour cloisonner ce qui doit être demandé ou non !

- Mettez en place des filtres dans votre code et testez-les en appelant les URLs directement dans votre navigateur :

    - un filtre par marque sur /api/parts (SearchFilter avec exact)
    - un filtre par catégorie (SearchFilter avec exact)
    - un filtre sur isAvailable (BooleanFilter)
    - un tri par prix (OrderFilter)

- Mettez en place une pagination de 12 par 12 sur les Parts

- Basculez l'API pour qu'elle fonctionne avec du JSON pur si on en fait la demande. Ça sera la base d'un projet Angular prochain ☺